

C Builds, Libraries, and Stream Buffering

Grounded in ECM2433 L08 files/streams and L10 program structure: stream buffering, setbuf, raw read/write, separate compilation, object files, libraries, and Makefiles.

What You Need To Know

1. Streams are buffered

A stream is an abstraction over input/output. Output is often stored in memory first, then written to the actual device later. This is buffering. A write call such as `printf` or `fprintf` may update the stream buffer without immediately showing text on screen or writing bytes to disk.

- Output to a terminal is commonly line-buffered, so a newline may flush it.
- Output may also be flushed when the buffer is full.
- The buffer size is implementation-dependent; lecture notes mention that 8192 bytes is common.
- Use `fflush(stdout)` or `fflush(fp)` when pending output must be forced out immediately.

```
printf("Step 1, ");
fflush(stdout);    /* force pending stdout text to appear now */
printf("Step 2\n"); /* newline may also flush terminal output */
```

2. setbuf and raw read/write

`setbuf` changes the buffering used by a stream. Passing `NULL` requests unbuffered stream I/O. Raw unbuffered I/O can also be done with lower-level `read` and `write`, which work with file descriptors rather than `FILE *` streams.

```
#include <stdio.h>

int main(void) {
    setbuf(stdout, NULL); /* request unbuffered stdout */
    printf("appears without waiting for a full buffer");
    return 0;
}
```

Do not mix up stream I/O and raw I/O. `fprintf`, `fgets`, and `fflush` use `FILE *`. Low-level `read` and `write` use integer file descriptors.

3. Separate compilation and object files

A larger C program is often split into multiple source files. Each `.c` file can be compiled into an object file with `gcc -c`. Object files contain compiled code, but they are not the final executable until the linker combines them.

```
gcc -std=c99 -c stack.c -o stack.o
gcc -std=c99 -c main.c -o main.o
gcc main.o stack.o -o program
```

- `stack.c` can compile to `stack.o` even if it has no `main` function.
- The final executable needs one `main` somewhere.
- Object files can be reused with different programs without recompiling the original source every time.

4. Linking and libraries

Linking resolves calls between object files and libraries. A library is a collection of object files packaged as one unit. Static libraries become part of the application at link time. Shared libraries are linked at runtime.

```
gcc -c myMaths.c -o myMaths.o
gcc myMaths.o -lm -o myMaths
```

In this example, `-lm` asks the linker to use the maths library, usually named `libm.so` or `libm.a`. Including `math.h` gives declarations to the compiler, but the linker still needs the library that contains the implementation.

5. Makefiles and dependencies

A Makefile records targets, prerequisites, and commands. The target is what you want to build. The prerequisites are the files it depends on. The command is how to build it. Commands must be indented with a tab.

```
CC = gcc
CFLAGS = -std=c99

program: main.o stack.o
<TAB>$(CC) $(CFLAGS) main.o stack.o -o program

main.o: main.c stack.h
<TAB>$(CC) $(CFLAGS) -c main.c -o main.o

stack.o: stack.c stack.h
<TAB>$(CC) $(CFLAGS) -c stack.c -o stack.o
```

```
clean:
<TAB>rm -f *.o program
```

The first target is the default target. `make` rebuilds a target only when the target is missing or one of its prerequisites is newer. If `stack.h` changes, both `main.o` and `stack.o` may need rebuilding because both depend on the header.

Practice Questions

Question 1 - trace buffered output:

```
printf("A");
printf("B");
fflush(stdout);
printf("C\n");
```

- 1 Explain what `fflush(stdout)` changes in this fragment.

Question 2 - choose the right I/O layer:

```
FILE *fp = fopen("out.txt", "w");
int fd = 1;
```

- 2 For each variable, say whether it belongs with `fprintf/fflush` or with `raw write`.

Question 3 - identify the missing build step:

```
gcc -std=c99 -c main.c -o main.o
gcc -std=c99 -c stack.c -o stack.o
```

- 3 Write the command that links these two object files into an executable called `program`.

Question 4 - header vs library:

```
#include <math.h>

int main(void) {
    return cosf(0.0f) == 1.0f ? 0 : 1;
}
```

- 4 The compile succeeds but linking reports undefined reference to `cosf`. Give the link command fix and explain why the header alone was not enough.

Question 5 - reuse an object file:

```
stack.o already exists
main.c changed
stack.c did not change
```

- 5 Give the compile/link commands that avoid recompiling `stack.c`.

Question 6 - complete a Makefile rule:

```
stack.o: stack.c stack.h
<TAB> /* command goes here */
```

- 6 Write the command for this rule.

Question 7 - decide what rebuilds:

```
program: main.o stack.o
main.o: main.c stack.h
stack.o: stack.c stack.h
```

- 7 If only `stack.h` changes, which targets should be rebuilt when making `program`?

Question 8 - find the Makefile bug:

```
program: main.o stack.o
gcc main.o stack.o -o program
```

- 8 This command line is indented with spaces. Explain the problem and the fix.

Question 9 - explain up-to-date behaviour:

```
$ make program
make: 'program' is up to date
```

- 9 What does this message mean about the target and its prerequisites?

Question 10 - complete the clean target:

```
clean:
<TAB> /* command goes here */
```

- 10 Write a suitable command for removing object files and the executable.

Mark Scheme

Attempt all questions before checking.

- 1 `fflush(stdout)` forces pending buffered stdout output to be written immediately. Without it, A and B might sit in the buffer until newline, full buffer, or program exit.
- 2 `fp` is a stream pointer for `fprintf/fflush`. `fd` is an integer file descriptor for raw write.
- 3 `gcc main.o stack.o -o program`.
- 4 Use `gcc main.o -lm -o program` or equivalent. `math.h` declares `cosf` for the compiler; `-lm` supplies the library implementation to the linker.
- 5 Compile only `main.c`: `gcc -std=c99 -c main.c -o main.o`, then link `gcc main.o stack.o -o program`.
- 6 `$(CC) $(CFLAGS) -c stack.c -o stack.o` or `gcc -std=c99 -c stack.c -o stack.o`.
- 7 `main.o` and `stack.o` are rebuilt because both depend on `stack.h`. Then program is relinked.
- 8 Makefile commands must start with a tab. Replace the leading spaces before `gcc` with a tab.
- 9 The target exists and is newer than, or at least not older than, its prerequisites, so no command needs to run.
- 10 Expected: `rm -f *.o program`.